NANA

tu jardín de manga

Informe

Aplicación en React con IA, Firebase y despliegue en Vercel

Materia	Desarrollo Frontend II · UADE · 2.º cuatrimestre 2026
Profesor	Joaquín Cerviño · Viernes, turno noche
Trabajo	Trabajo Práctico 2 · Versión final v1.2
Apellido y nombre	Retrive, Serena
L.U.	17359595
App publicada	jardin-de-manga.vercel.app
Repositorio	github.com/serenaretrive02/jardin-de-manga
Fecha	13 de noviembre de 2026

Trabajo Práctico 2 · Desarrollo Frontend II · UADE Prof. Joaquín Cerviño · Viernes, turno noche · 2.º cuatrimestre 2026 · Versión final: **v1.2**

Entrega	Enlace
App publicada (Vercel)	https://jardin-de-manga.vercel.app
Repositorio (GitHub)	https://github.com/serenaretrive02/jardin-de-manga
Bitácora del proceso (8 versiones, con capturas)	bitacora.md
Especificación (requisitos, casos de uso, sprints)	especificacion.md
Guía para el oral (dónde está cada tema en el código)	guia-oral.md
Video de demostración	*pendiente: agregar el enlace*

1. La idea y su público

Nana es una app para llevar el registro de los mangas y animes que seguís. **Cada título es una flor**: la especie depende del género (romance → rosa, acción → girasol, fantasía → tulipán...) y la planta **crece a medida que avanzás capítulos** (semilla → brote → creciendo → en flor). Si la abandonás, se marchita. **Cuando la historia termina para vos, la puntuás de 1 a 5 sakuras** (flores de cerezo), podés dejar una reseña corta y queda anotada en tu **Diario de lecturas**, como en Letterboxd. Además, Nana es una **comunidad**: permite armar clubes de lectura con amigas, repartirse misiones y organizar eventos.

Necesidad que resuelve: quienes leen manga o miran anime siguen muchas series a la vez y se olvidan por dónde iban. Nana lo guarda y lo muestra de forma visual y motivadora, y suma la experiencia de leer en grupo.

Público: lectoras y lectores de manga y espectadores de anime, sobre todo jóvenes, que no quieren crear una cuenta ni completar formularios largos (el ingreso es anónimo).

Por qué se ve así: estética **pixel art** de videojuego retro (ventanas tipo "programa", paleta lima y rosa, sombras duras), porque conecta con un público gamer y otaku y refuerza la idea de "cuidar un jardín" como en un juego. Las 9 flores, la abeja, las enredaderas y el pasto están **dibujados con código**: son arrays de texto que React convierte en píxeles. En la última iteración (v1.1) se sumó un toque **kawaii** inspirado en planificadores tiernos: más espacio para que los textos respiren, esquinas "redondeadas en píxeles", sombras rosadas y enredaderas de cerezo, siempre con la misma paleta. En la versión final se sumaron un **gatito dormilón** pixelado y una **canción de fondo original** tipo cajita musical, que se apaga con el botón Sonido. El diseño se refinó en ocho versiones a partir de referencias visuales y de las capturas de cada prueba (ver la bitácora).

2. Funciones implementadas (todas con CRUD)

Módulo	Crear	Consultar	Modificar	Eliminar
Jardín (plantas)	Plantar, a mano o buscando en la API	Ver el jardín con filtros, buscador, orden y "Mi ramo"	Regar +1, cambiar el estado, editar	Compostar (con confirmación)
Puntuación y reseña (v1.2)	Tocar una sakura o escribir la reseña (solo si la planta terminó o se abandonó)	En la tarjeta, el promedio de "Mi ramo" y el orden "Mejor puntuadas"	Tocar otra sakura o editar el texto	Tocar la misma sakura o vaciar la reseña: el campo se borra con `deleteField()`
Diario de lecturas (v1.2)	Al terminar o abandonar una obra se anota sola con la fecha (`terminadoEn`)	Pestaña Diario: lecturas por mes, resumen, gráfico de puntuaciones, filtros y 4 órdenes	Puntuar, reseñar y corregir la fecha	"Borrar reseña" (con confirmación); si vuelve a "Leyendo", sale del diario y la fecha se borra
Perfil	Apodo y flor-avatar (9 flores)	Ver el perfil y el de los miembros	Editar	Eliminar perfil
Clubes (grupos)	Crear club (con código e invitación)	Mis clubes, miembros	Editar club; unirse, salir o quitar miembros	Eliminar club (borra sus misiones y eventos)
Misiones (tareas de 6 tipos)	Crear y **asignar: solo la administradora**	Tablero Kanban, "solo mis misiones"	Mover de columna (la persona asignada); editar (la administradora)	Eliminar (la administradora)
Eventos	Cualquier miembro	Próximos y pasados	Editar (quien lo creó o la administradora); anotarse o borrarse	Eliminar (quien lo creó o la administradora)

Además:

- **Compartir** clubes, misiones y eventos (menú del celular o portapapeles), con un enlace de invitación que completa el código solo.
- **Favoritas sin sección aparte:** el orden "Mejor puntuadas" deja arriba las que más te gustaron. La lista "quiero leer" ya existe: es el estado **Pendiente**.
- **Herbario** con las 9 flores, su significado (*hanakotoba*) y cuántas tenés.
- **Guía flotante** de cómo crece la flor.
- **Nana IA**, una asistente con forma de abeja que responde paso a paso sobre la app.
- **Música de fondo original** (cajita musical generada con Web Audio, sin archivos) y **sonidos** retro. Todo se apaga con un solo botón.
- **Gatito dormilón** sobre el pasto: si lo tocás, maúlla y aparece un corazón.
- **Cerrar sesión**, que empieza un jardín nuevo y avisa antes.

Experiencia de uso (consigna, punto 6):

- validaciones con mensajes debajo de cada campo (título, capítulos, reseña, fecha del diario);
- barras de carga pixeladas;
- avisos de éxito y de error en cada operación;

- diseño probado en compu y en celular (390 px), con botones de al menos 44 px.

3. Información guardada en Firebase

- **Authentication (anónimo):** identifica a cada visitante sin pedirle registro.
- **Cloud Firestore:**

Ruta	Qué guarda	Quién lo ve
`users/{uid}/plantas/{id}`	título, tipo, género, estado, capítulos leídos y totales, portada y número de Kitsu (opcionales), puntuación de 1 a 5, reseña de hasta 280 caracteres y fecha del diario `terminadoEn` (opcionales, v1.2), fecha de creación	Solo su dueña
`users/{uid}`	apodo y flor-avatar	Cualquier visitante identificado (para mostrar nombres en los clubes)
`clubes/{CÓDIGO}`	nombre, descripción, administradora, miembros	Solo sus miembros
`clubes/{CÓDIGO}/misiones/{id}`	título, tipo, a quién está asignada, estado, fecha límite	Miembros del club
`clubes/{CÓDIGO}/eventos/{id}`	título, tipo, fecha, lugar, asistentes	Miembros del club

Seguridad:

- Las **reglas de Firestore** (`firestore.rules`) controlan quién puede hacer qué y que cada dato tenga la forma correcta, y solo aceptan campos conocidos.
- **Solo la administradora puede crear o asignar misiones.** Se probó llamando directo a Firebase como miembro: la respuesta fue **403 (rechazado)**.
- La **puntuación** solo se acepta si es un entero de 1 a 5, y la **reseña** si tiene de 1 a 280 caracteres. La fecha del diario solo se acepta con formato AAAA-MM-DD. Se probó guardar 9 sakuras, una reseña de 400 caracteres y la fecha "ayer" llamando directo a Firebase: **403 (rechazado)**.
- Las **llaves de conexión** están en `.env` , que **no está en GitHub**. En Vercel se cargaron como variables de entorno.

Dónde se configura Firebase: en `src/firebase.js` , que lee las variables `VITE_FIREBASE_…` del `.env` con `import.meta.env` . Las operaciones están en `src/services/` (`plantas.js` , `perfiles.js` , `clubes.js` , `misiones.js` , `eventos.js`).

4. Herramientas utilizadas

Visual Studio Code · Node.js + npm · React + Vite · Firebase (Authentication y Cloud Firestore) · Git + GitHub · Vercel · API pública de **Kitsu** (búsqueda de obras, sin clave) · **Claude Code** (asistente de IA de Anthropic) · Google Chrome con navegador automático (para pruebas y capturas).

5. Uso de inteligencia artificial

Se usó **Claude Code**, que trabaja directamente sobre la carpeta del proyecto, para:

- pensar y comparar ideas;
- generar código siguiendo las guías de clase;
- revisar la consigna y la rúbrica;
- configurar Firebase y Vercel desde el navegador;
- probar la app con un navegador automático;
- redactar la documentación.

Cómo se verificó lo que hizo la IA: cada versión se probó con **pruebas automáticas** (de 7 a 26 pasos por versión, incluida una con dos usuarias a la vez; 152 en total en la versión final v1.2), se revisaron las **capturas** en compu y en celular, se comprobaron los datos en la **consola de Firestore** y se repitieron las pruebas **sobre la URL de Vercel**. Gracias a eso se encontraron y corrigieron más de 25 problemas (ver la bitácora). Los pasos que implicaban cargar llaves los hizo la autora, y los que publicaban algo (por ejemplo, las reglas de la v1.2) se hicieron con su autorización.

Ejemplos representativos de instrucciones dadas a la IA:

1. *"Antes de crear cualquier aplicación, pensá ideas sumamente creativas y originales según las cosas que me gustan (Minecraft, LoL, manga, anime, flores), que permitan hacer cada acción que pide la consigna, pero que no sea una típica."*
2. *"Todo lo que hagamos tiene que ser lo que vimos en clase (Visual Studio Code, GitHub y lo de los PDF)."*
3. *"Vayamos con la 0.2. Me gustaría que el diseño sea tipo pixel art"*, con tres imágenes de referencia.
4. *"Cambiemos el título a Nana."* (v0.3: la IA investigó por qué la API planeada no respondía y propuso otra).
5. *"Chequeá que se cumpla todo lo que pide la consigna y la rúbrica [...]. Quiero sumar una comunidad, grupos de lectura, CRUD de tareas, eventos y grupos [...]. El administrador es el único rol que puede asignar tareas."*
6. *"Me gustaría que los avatares sean distintos tipos de flores, una leyenda de cómo crece la flor, una abeja revoloteando y una asistente Nana IA [...]. Para la versión final: filtros, significado de cada flor, sonidos [...]. Si ves algo que no cumple la consigna, eliminalo."*
7. *"Se me ocurre agregar favoritos para anotar los mangas que quiero ver, o puntuarlos como una especie de Letterboxd, pero no me gustaría saturar el sitio [...]. Recordá que cada acción debe poder agregarse, eliminarse y seguir el CRUD, y tener la metodología ágil."* (v1.2: la IA notó que "quiero leer" ya existía como estado Pendiente y se eligió la puntuación con sakuras).
8. *"Me gustaría que en algún lado queden guardadas las puntuaciones que hice y los mangas que leí [...] como en Letterboxd. [...] Aumentá el padding del texto en crear clubes [...]. Agregá una canción kawaii de fondo que se pueda mutear con el botón de sonido. [...] Hacé una revisión general para que todo cumpla con la consigna."* (v1.2: el Diario reutiliza los mismos datos en lugar de duplicarlos; la canción es original y generada con código).

Decisión sobre Nana IA: conectar un modelo de IA externo exige una clave privada, y la consigna prohíbe incluir claves privadas en el código del navegador. Por eso Nana IA responde con una **base de conocimiento propia** de la app (`src/data/conocimiento.js`). No expone claves, no tiene costo y responde solo sobre Nana.

6. Dificultades principales y cómo se resolvieron

(El detalle de cada una, con capturas, está en la bitácora.)

- El `.gitignore` de Vite **no protegía el** `.env` : se agregó antes del primer commit.
- Windows bloqueaba `npm` (*ExecutionPolicy*): se usó el comando que indicó el profesor.
- Se borró `.env.example` por error: se restauró, y desde entonces se revisa la lista de cambios antes de cada commit.
- La API planeada (Jikan) estaba **caída**: se investigó y se cambió por Kitsu, y la búsqueda quedó como ayuda opcional.
- Diseño en celular: textos cortados y formularios que se salían de la pantalla. Se corrigió con tarjetas horizontales y `minmax(0, 1fr)` .
- **Roles:** para que nadie pueda saltearse la interfaz, el permiso de la administradora se controla en las reglas de Firestore.
- Tipografía: la fuente pixelada no tenía "Í"/"Ó" en mayúscula y leía "flor" como "Aor": se ajustaron las fuentes.
- Fechas: los eventos se calculaban en hora UTC: se pasó a hora local.
- El aviso flotante **tapaba botones** que quedaban debajo: ahora deja pasar los clics.
- En el pulido final (v1.1), una regla nueva de diseño "pisó" la del celular y el jardín quedó en dos columnas: se detectó en las capturas y se corrigió.
- En la v1.2, Firebase **rechazaba la puntuación** porque las reglas solo aceptan campos conocidos. Se publicaron las reglas nuevas **antes** de subir el código. Para **eliminar** una puntuación se usó `deleteField()` , que borra el campo en lugar de guardar un 0.
- Con 4 pestañas, en el celular "Herbario" quedaba **cortada**, aunque la prueba automática daba "sin scroll horizontal" (la página recorta lo que sobra). Se vio en la captura, se corrigió y se mejoró la prueba.
- Los navegadores **bloquean el sonido automático**: la música empieza con el primer toque de la persona en la página.

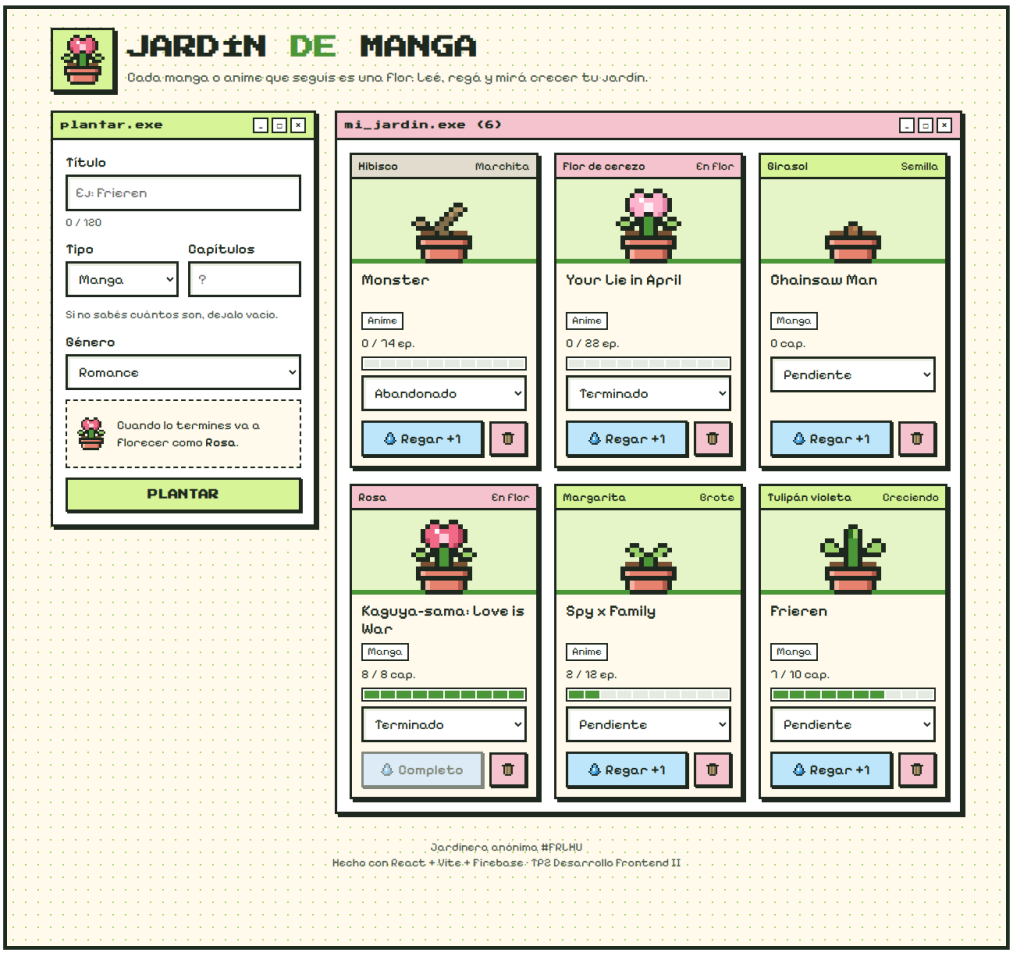
7. Capturas

Evolución del diseño (de la v0.1 a la versión final)

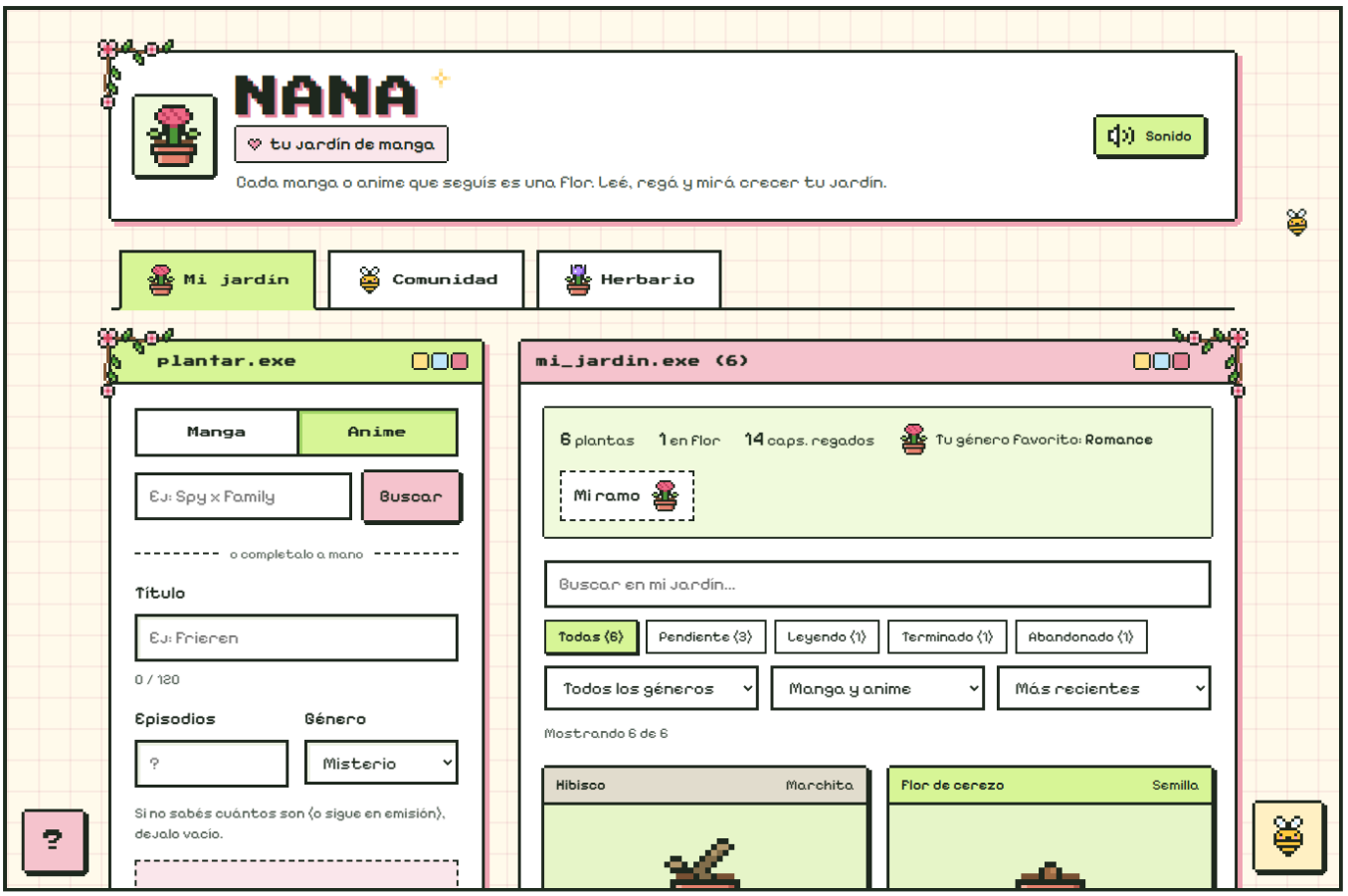
La interfaz no es el primer resultado de la IA: se refinó en ocho versiones. Estas son cuatro etapas clave (el detalle de cada cambio está en la bitácora).



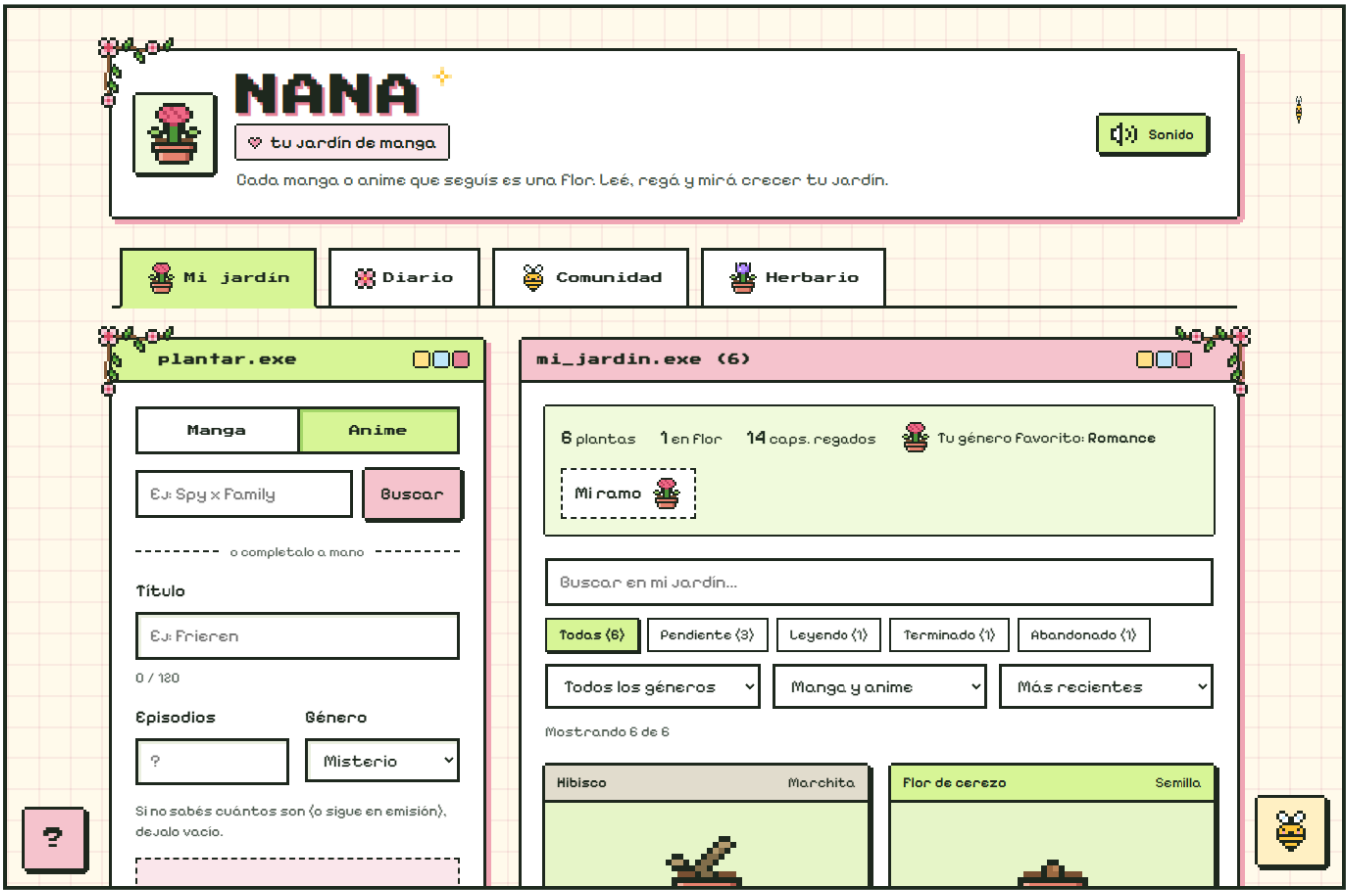
v0.1 — MVP: el CRUD funciona con Firestore, pero el diseño es básico.



v0.2 — Identidad pixel art: ventanas tipo "programa", flores dibujadas con código, validaciones y avisos.

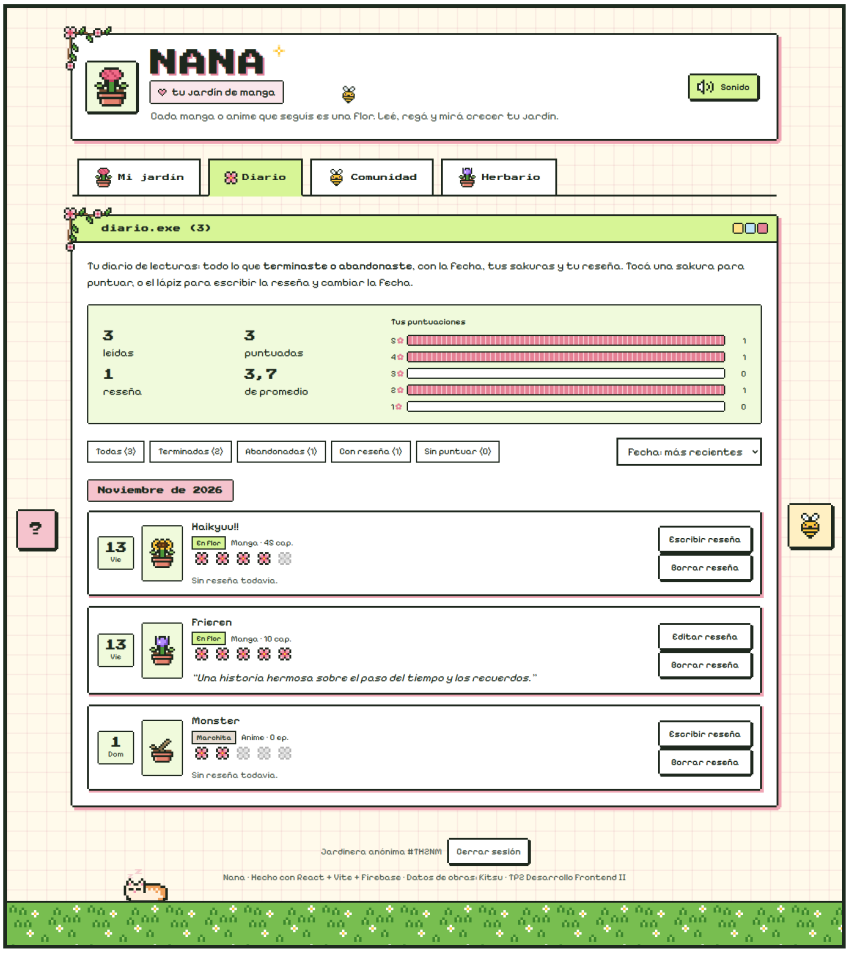


v1.1 — Pulido kawaii: más espacio, esquinas pixeladas, enredaderas de cerezo y pasto con margaritas.

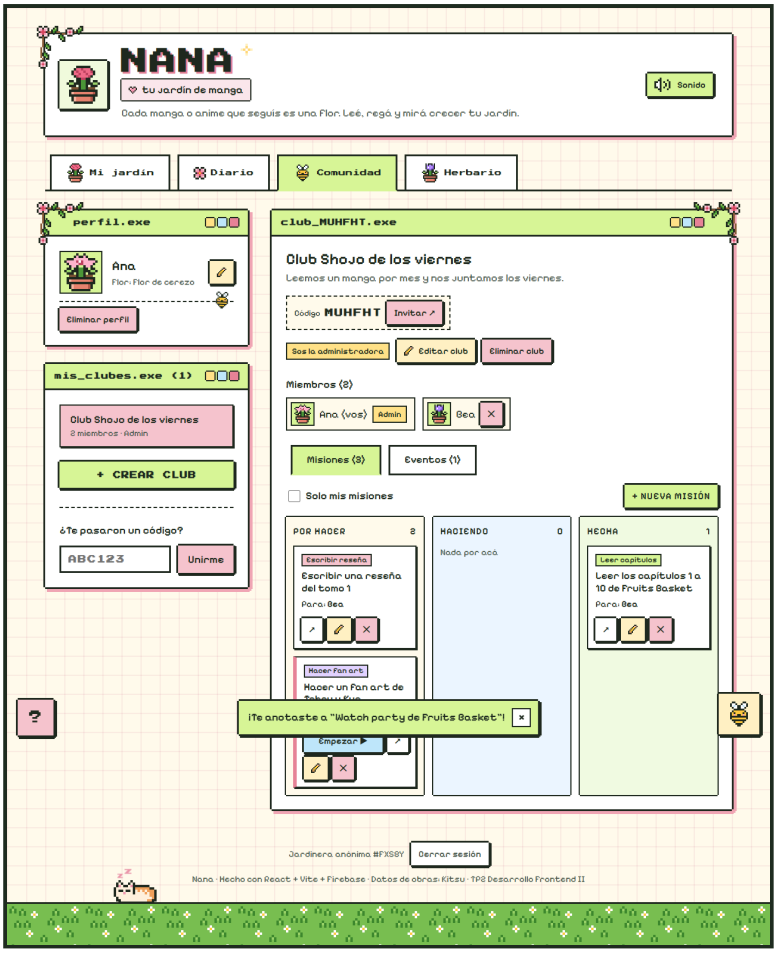


v1.2 (final) — Cuatro pestañas (Mi jardín, Diario, Comunidad, Herbario), "Mi ramo" con estadísticas y filtros del jardín.

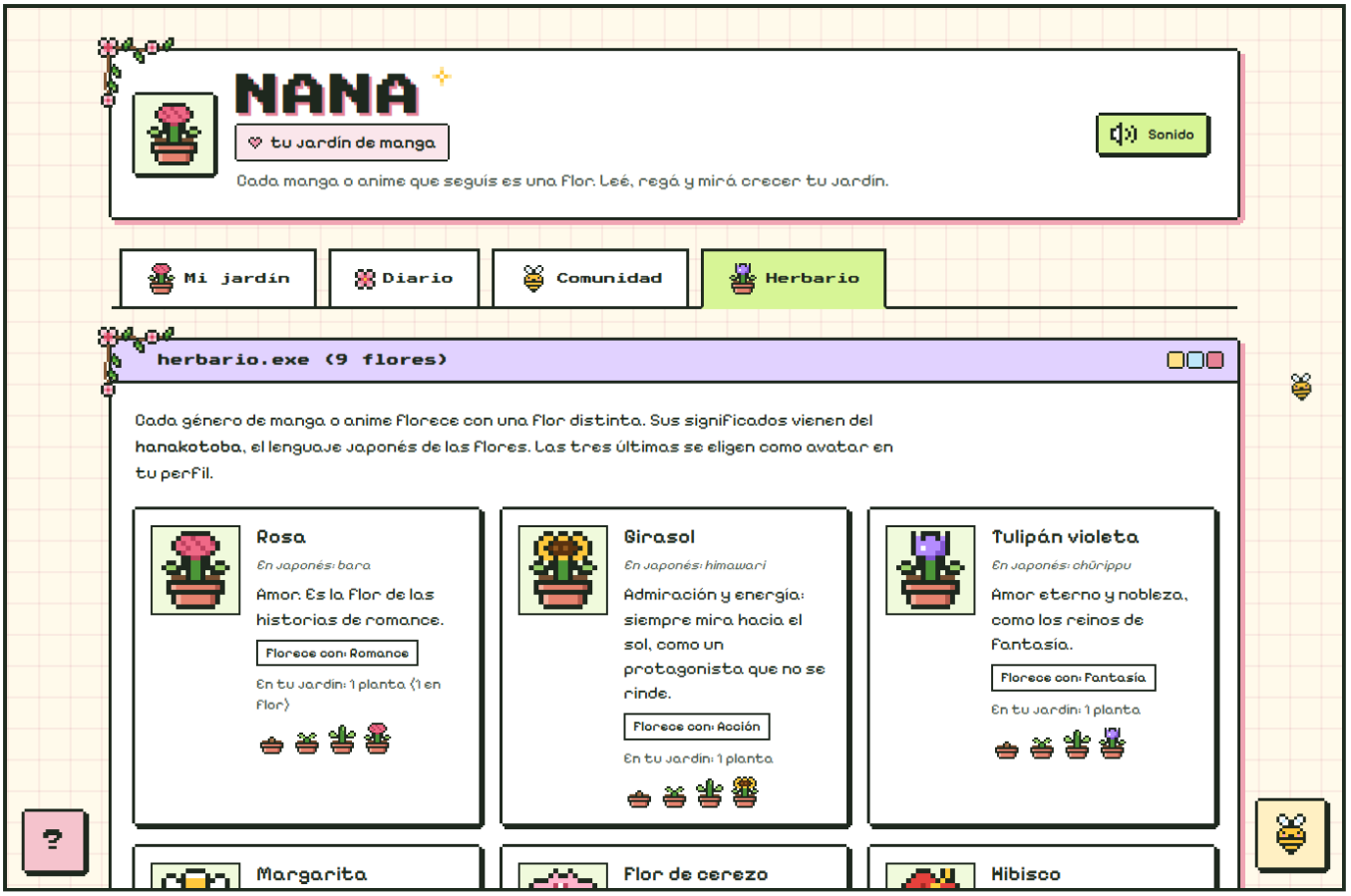
La versión final



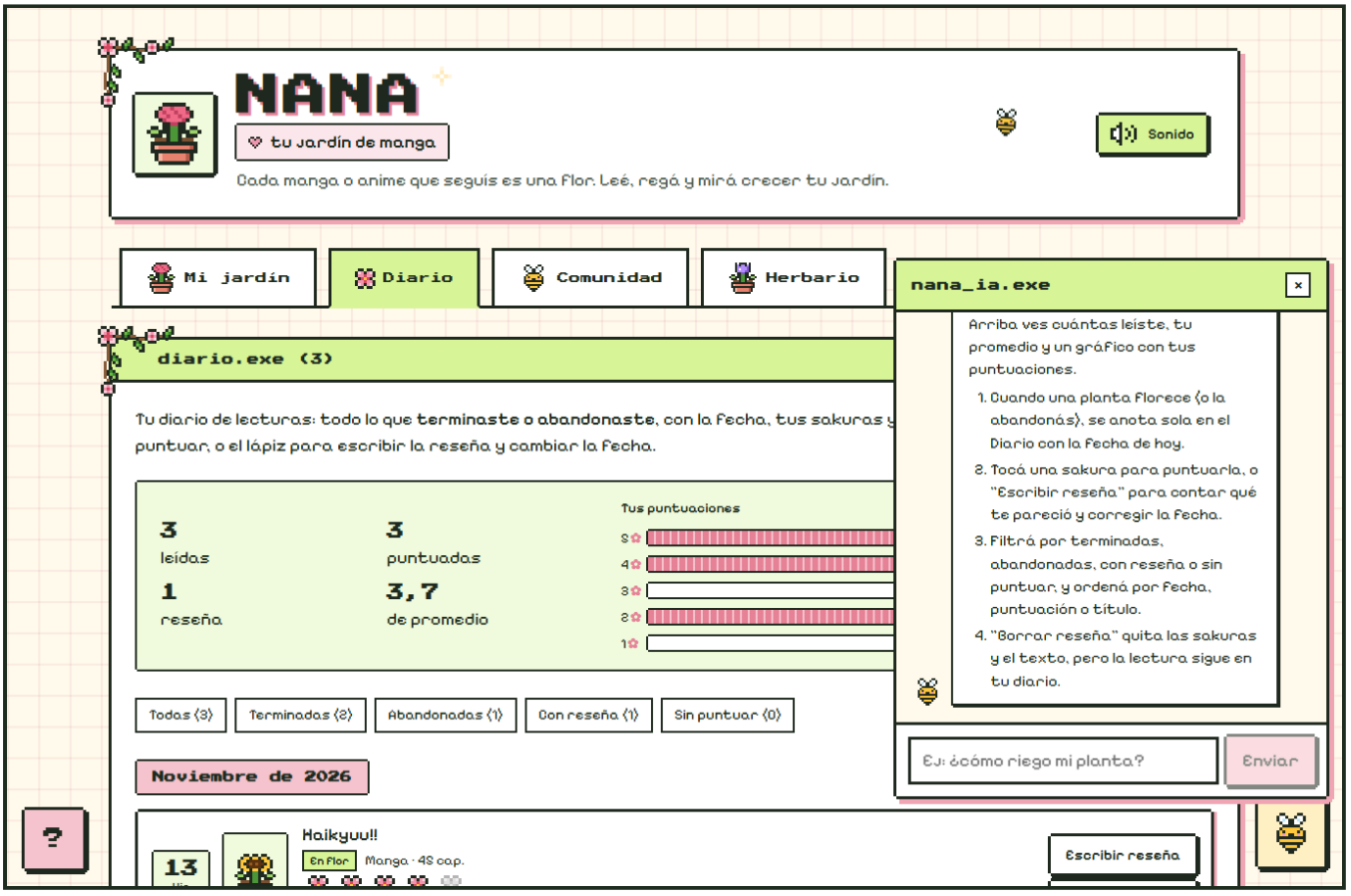
Diario de lecturas (v1.2): lo terminado o abandonado, por mes, con resumen, gráfico de puntuaciones, sakuras y reseñas.



Comunidad, vista de la administradora: código para invitar, miembros, misiones en un tablero Kanban y el botón "+ Nueva misión" (solo ella puede asignarlas). Abajo se ve un aviso de éxito y el gatito sobre el pasto.



Herbario: las 9 flores con su significado en japonés (hanakotoba).

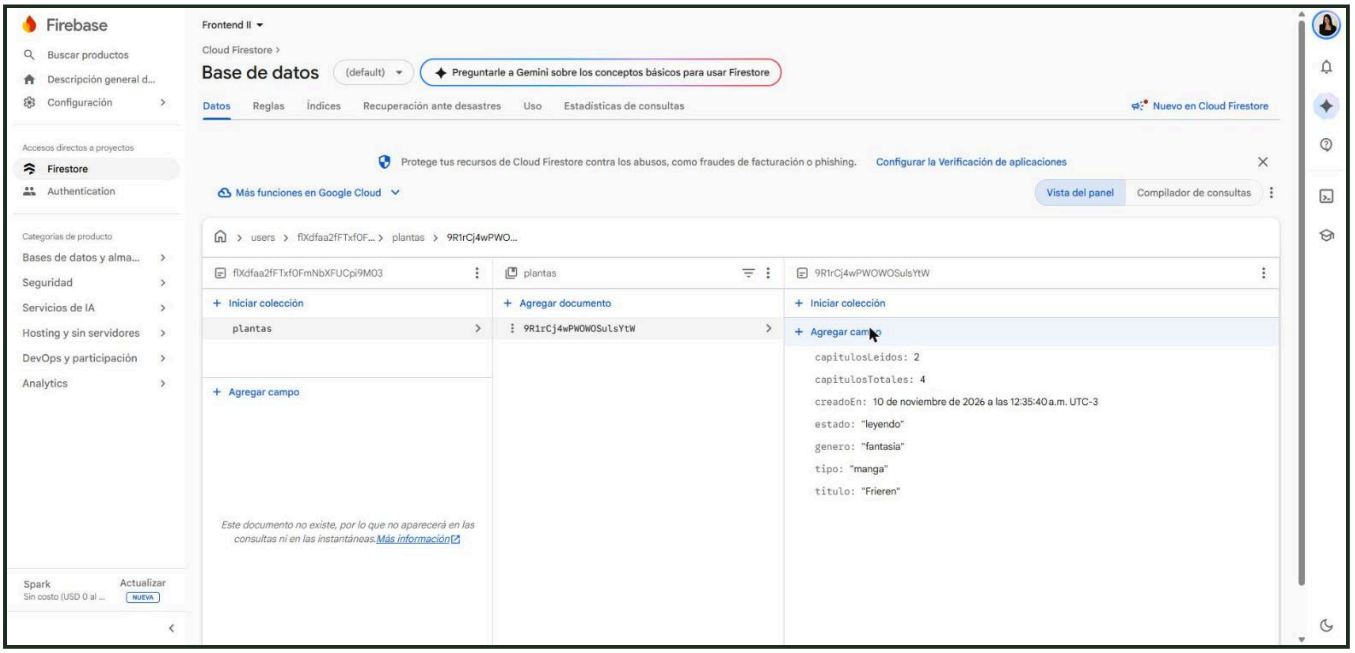


Nana IA responde paso a paso, solo sobre la app y sin claves externas.



La misma app en un celular (390 px): una columna y las 4 pestañas completas.

Firebase



Un registro creado desde la app, visto en Firestore → Datos.



Las reglas publicadas en Firebase: solo aceptan campos conocidos y con la forma correcta.